

HTTP - PROTOCOLE

- **Couche Application** (pas transport)
- Protocole **sans etat/memoire** = chaque comm. independante, pas de session
- Mode **requete/reponse** : pour chaque req = **forcement** une reponse
- Donnees **MIME** | Port **80** | Connexion **TCP**
- **Bidirectionnel** mais **half-duplex** (req puis rep, pas simultane)
- Support **caches** et **proxy** | Negociation de fonctionnalites
- **Cookies** = suivi de session (remedier au sans-etat)
- HTTP est **leger** : ne transporte que du **texte**

Methodes HTTP

GET	Recuperer contenu URL sans envoyer de body
POST	Envoyer donnees dans le body (formulaires)
HEAD	Comme GET, en-tete seulement (pas de body)
PUT	Enregistrer/maj document (HTTP 1.1+)
DELETE	Effacer ressource (HTTP 1.1+)
OPTIONS	Options de comm. disponibles (HTTP 1.1+)
TRACE	Echo de la requete recue (HTTP 1.1+)

QCM : GET ne transmet PAS de donnees dans le body. HEAD = en-tete sans corps. PUT/DELETE/OPTIONS = HTTP 1.1+

Codes reponse

- **1xx** Info (100 Continue) | **2xx** Succes (**200** OK, 201 Created, 204 No Content)
- **3xx** Redirect (301 Moved, 302 Found, 304 Not Modified)
- **4xx** Erreur Client (400 Bad Req, 401 Unauth, 403 Forbidden, **404 Not Found**)
- **5xx** Erreur Serveur (**500** Error, 502 Bad Gateway, **503** Unavailable)

QCM : 200 = succes. 404 = Not Found. 500/503 = PAS des succes!

HTTP VERSIONS (IMPORTANT)

	HTTP/1.0	HTTP/1.1	HTTP/2	HTTP/3
Transport	TCP	TCP	TCP	QUIC (UDP)
Connexion	Fermee apres chaque req	Persistante (keep-alive)	1 seule TCP	1 seule QUIC
Requetes	1 a la fois	Pipeline (sequentiel)	Multiplexage (parallele)	Multiplexage
En-tetes	Texte brut	Texte brut	HPACK (compresses)	QPACK
Push	Non	Non	Server Push	Possible
Securite	Optionnel	Optionnel	Recommande TLS	TLS 1.3 obligatoire

TCP vs UDP / QUIC

- **TCP** = fiable, connexion, controle flux. Probleme : **head-of-line blocking** (1 paquet perdu bloque tous les autres)
- **UDP** = rapide, sans connexion, pas de garantie d'ordre
- **QUIC** = protocole au-dessus d'**UDP**, ajoute fiabilite + TLS 1.3 integre, sans head-of-line blocking

HTTP/2 vs HTTP/1.1

- HTTP/2 : **multiplexage** = plusieurs req/rep en parallele sur 1 connexion TCP
- Compression en-tetes **HPACK** = moins de trafic reseau
- **Server Push** = serveur envoie ressources avant la demande
- MAIS : toujours TCP = head-of-line blocking au niveau transport

HTTP/3 vs HTTP/2

- Base sur **QUIC (UDP)** au lieu de TCP = supprime head-of-line blocking transport
- **Full-duplex natif complet** = echanges independants client/serveur
- Perte de paquet sur 1 flux n'empeche PAS les autres flux
- **TLS 1.3 obligatoire** = securite integree, moins de latence
- Connexions plus rapides (0-RTT possible)

QCM : HTTP fonctionne avec TCP. HTTP/3 peut utiliser UDP (via QUIC). En HTTP/2+ la connexion reste ouverte (multiplexage), PAS fermee apres chaque req.

EN-TETES + COOKIES

En-tetes requete

- **Host** : machine+port serveur | **Accept** : types MIME acceptes
- **Accept-Encoding** : br, gzip, deflate | **Accept-Language** : fr, en
- **User-Agent** : info navigateur | **Cookie** : cookies stockes
- **Authorization** : Bearer token (OAuth, JWT) | **Referer** : URL d'origine
- **Connection: keep-alive** = connexion persistante (HTTP/1.1)
- HTTP/2+ : Connection plus utilisee (automatique)

En-tetes reponse

- **Content-Type** : type MIME | **Content-Length** : taille en octets
- **Set-Cookie** : creer cookie | **Last-Modified** : date modif (caches)
- **Cache-Control** | **ETag** | **Allow** : methodes autorisees

Cookies

- Serveur : Set-Cookie: NOM=VAL; Path=/; Secure; HttpOnly
- Client : Cookie: NOM1=VAL1; NOM2=VAL2
- **Secure** = HTTPS only | **HttpOnly** = **interdit acces JavaScript**
- **Path** = chemin sur le serveur pour lequel le cookie est valide

QCM : HttpOnly = interdit acces JS (PAS limiter HTTPS, PAS definir chemin). HTTPS = HTTP + TLS/SSL, port 443

LIMITES HTTP + WEBSOCKET

- Limites : **half-duplex**, verbeux (headers chaque req), pas de push natif
- Solutions : Long polling, Streaming, SSE, **WebSocket**

WebSocket

- Canal **full-duplex** par-dessus TCP/UDP
- URI : ws(s)://host/path
- Etabli via HTTP (**Upgrade: websocket**) depuis HTTP/1.1
- Apres upgrade **HTTP n'est plus utilise**
- Fermeture cote client OU serveur | Ex: Socket.io (Node.js)

JAVASCRIPT

- **Faiblement type, multi-paradigme** (procedural, OO, fonctionnel), **dynamique**
- Standard **ECMAScript** (ES6/2015) | Execution **haut en bas**
- Chaque onglet = environnement **separe** (securite)

Variables

	var	let (ES6)	const (ES6)
Portee	Fonction/glob	Bloc {}	Bloc {}
Re-decl	Oui	Non	Non
Hoisting	Oui (undef)	Non (RefErr)	Non
Re-assign	Oui	Oui	Non

- Var globales accessibles via **window** (objet fenetre navigateur)
- **let** = portee de bloc (ES6). var = mauvaise pratique

- Types
- **Number** (int+float) | **String** ("" " ``") | **Boolean** | **Array** | **Object**
 - **null** = absence intentionnelle | **undefined** = non initialise
 - **null** == **undefined** **true** | **null** === **undefined** **false**
 - Template : `Nom: \${nom}`

- Operateurs
- **===** compare **type ET valeur** (stricte) | **==** valeur seulement
 - **typeof** type | **instanceof** test instance | **new** creer objet
- QCM : == compare seulement valeur, === compare type ET valeur. == n'est PAS obsolete.

- Sortie
- **.innerHTML** = modifier contenu HTML element | **.textContent** (safe)
 - **document.write()** **ecrase tout apres chargement!**
 - **alert()** = boite dialogue popup (**PAS la console!**)
 - **console.log()** = console dev F12
 - **window.print()** = **imprimer** la page (PAS affichage!)
- QCM : alert() affiche popup PAS console. innerHTML pour modifier HTML. createElement() = methode de **Document**

- Fonctions
- Classique : **function f(a) { return a*2; }**
 - Flechee : **const f = a => a*2;** (retour implicite sans {})
 - **this** flechee = **contexte englobant** (different de fonction classique!)
 - Flechee sans {} ni return = **retourne implicitement** l'expression
- QCM : Flechee sans {} retourne implicitement. this flechee = contexte englobant (PAS l'appelant)

- String / Array / JSON
- Str: **.length** **.indexOf()** **.slice()** **.toLowerCase()** **.replace()**
 - Arr: **.push()**/**.pop()** fin | **.unshift()**/**.shift()** debut
 - **.sort()** **.join()** **.split()** **.forEach(fn)** **.toString()**
 - **JSON.stringify(obj)** obj→texte | **JSON.parse(txt)** texte→obj

- Boucles / Erreurs
- **for**, **while**, **do..while**, **for..in** (props), **for..of** (vals), **.forEach()**
 - **try{}catch(e){}finally{}** | **throw**
 - **"use strict"** : interdit var non declaree, double param

POO JAVASCRIPT

- Objet = **proprietes + methodes** | **this** = objet courant
- **ES5 ne supporte PAS les classes** classiques → utilise **prototypes**

- Constructeur + Prototype
- ```
function Rect(l,h) { this.l=l; this.h=h; }
Rect.prototype.aire = function() {
 return this.l * this.h;
}; // PARTAGEE entre toutes les instances
```
- Prototypes = methodes communes partagees** (economie memoire). Heritage EST supporte. On PEUT mutualiser les methodes.

- Heritage
- ```
function Chien(nom, race) {
  Animal.call(this, nom); this.race = race;
}
Chien.prototype = Object.create(Animal.prototype);
```

- Classes ES6
- ```
class Uv { constructor(n) { this.nom=n; } }
class UvS extends Uv {
 constructor(n,t) { super(n); this.type=t; }
}
```

**ATTENTION** : class PAS l'unique solution (prototypes aussi). Pas de hoisting classes. P = e leve("a","b") sans **new** = FAUX!

- Methodes Object
- **Object.keys/values/entries(obj)** | **Object.fromEntries()**
  - **Object.assign({},o1,o2)** | **Object.create(proto)**
  - **Object.freeze()** gele | **Object.seal()** modif OK pas ajout

ASYNCHRONE (IMPORTANT)

- JS = **monotheade** = 1 seul thread pour tout
- **Event loop** = mecanisme qui delegue les operations lentes (E/S) au systeme, puis execute les callbacks quand c'est pret
- Operations bloquantes (HTTP, fichiers, BD) = **doivent etre async**
- La **pile d'appels** (call stack) execute les operations synchrones
- Les ops async sont gerees par l'event loop → delegue au noyau OS

3 mecanismes async

|                    |                                                    |
|--------------------|----------------------------------------------------|
| <b>Callback</b>    | <b>fn(param, function(res){...})</b>               |
| <b>Promise</b>     | <b>fn().then(d=&gt;...).catch(e=&gt;...)</b>       |
| <b>Async/Await</b> | <b>const v = await fn();</b> (dans async function) |

- Promise
- ```
new Promise((resolve, reject) => {
  ok ? resolve(val) : reject("err");
}).then(d => ...).catch(e => ...);
```
- **resolve** = succes → **then()** | **reject** = echec → **catch()**
 - **Callback Hell** = callbacks imbriques = illisible → Promises/await
 - Promises **reduisent le code imbrique** et gerent les erreurs en fin de chaine
 - **setTimeout(callback, delay)** = simule traitement async

- Async/Await
- ```
const asyncFn = async function(delay) {
 const value = await maPromise(delay);
 console.log(value);
};
```
- **await** attend le resultat d'une Promise
  - Ne peut etre utilise que dans une fonction **async**

QCM : Node.js = programmation asynchrone VRAI. Event loop = non bloquant. Variable lue avant fin async = **undefined**

